

Docker

- (i) Virtualization Software (virtualize the OS Application Layer)
- (ii) Packages application with all necessary configs, dependencies System tools and runtime.

Commands

`docker run -d -p {host-port}:{container-port} Image-name`

`docker images`

`docker ps`

`docker stop Image-id`

`docker stop Image-id`

`--name =` Assign a name to the container

`docker pull`

`docker logs image-id`

Docker Registry

- ✓ A service provides storage
- ✓ Collection of repositories

Docker Repository

Collection of related images with same name but different versions

Docker Compose:

It is tool for defining and running multiple container docker application

In docker compose file, you will write configure & maintain your application's services.

With a single command, you can run all the services.

How to write docker compose yaml:

compose.yaml

version: 3.1 → This should be compatible with compose installed locally.

services: (list all the services you want to run)

mongodb: => container name for your image

image: mongo:5.6

ports:

- 27017 : 27017
(host) (container)

environment:

- MONGO_USERNAME = admin
- MONGO_PASSWORD = secret

service 2:

image:

ports:

- :

environment:

- env-var1: admin
- env-var2: pwd

By default, docker compose sets up a single network for the app.

communication is done via container_name.

Command:

`docker-compose -f compose.yaml up`

↳ builds, creates & starts the defined containers.

And also picks up any changes and recreating the containers if the changes are made after the creation.

`docker-compose -d -f compose.yaml down`

LIMITATIONS:

* Well suited for local development & small scale deployments

* Designed to run the containers on single host

* Engg still needs to operate the containers manually.

A lot of operational efforts to run containers on large-scale

PROS:

→ Central Place

→ perfect for microservice apps

→ Scaling to 1000s of containers

Kubernetes Rescue!